DEPARTMENT OF
COMPUTER SCIENCE

ALGORITHMS II

DR JEREMIAH O. BANDELE

PhD Electrical and Electronic Engineering

University of Nottingham

LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- Develop an algorithm using the algorithm development steps.
- Describe algorithm complexity.
- Discuss algorithm characteristics and design considerations.

WRITING AN ALGORITHM

Consider the problem statement below;

Problem statement: For a given set of numbers (obtained from user input), determine the maximum number.

Using the algorithm development steps, we will now write an algorithm to solve a problem.

WRITING AN ALGORITHM: STEP 1

Step 1: Problem description (For a given set of numbers provided by system users, determine the maximum number)

This is a simple problem so it is easy to describe. We only need to accept some inputs from the user and then, calculate the maximum number. Any issues here?

- Are there unstated assumptions in the description?
- Is the description ambiguous or incomplete?

Writing an algorithm: step 2

Step 2: Problem analysis

Starting point considerations include;

- What type of data is the program expecting from the user?
- What formula will be used to determine the maximum number?

Ending point considerations include;

- What type of result are we expecting from the algorithm?

WRITING AN ALGORITHM: STEP 3

Step 3: High-level algorithm development

An high level algorithm focused on the major solution points, is provided below;

```
Declare and initialize variables
Input numbers (prompt user and accept input)
Determine maximum number and output result
```

The algorithm can be revised as shown below;

```
Declare and initialize variables
Input numbers  - -  compare each input with the
        Current maximum to know which is higher
Identify maximum number and output result
```

Writing an algorithm: step 4

Step 4: Algorithm refinement

We can now refine line 2 of the high-level algorithm as shown below;

```
Declare and initialize variables
Loop until the user enters sentinel value
        prompt user to enter a number
        allow user to type in a number
        compare the number with the maximum
        add 1 to a counter
Identify maximum number and output result
```

Writing an algorithm: step 5

Step 5: Algorithm review

The last step is to review the algorithm by considering the points below:

- Is this algorithm only useful for determining the maximum of some set of numbers?
- Can we reduce the complexity of the algorithm?
- Can other problems be solved with the algorithm?

Algorithm analysis

- Priori analysis: Does not directly affect the implementation of an algorithm. These analyses are carried out before implementing an algorithm in a particular programming language.
- Posterior analysis: Carried out after an algorithm has been implemented in a particular programming language. Deals with issues such as the space and running time requirements of an algorithm.

Mid-lesson question

1. How do algorithms influence our lives and shape our decision-making processes?

ALGORITHM COMPLEXITY

The complexity/performance of an algorithm can be analysed in terms of time complexity and space complexity.

- Time complexity deals with the time required by an algorithm to complete its execution.
- Space complexity deals with the amount of computer space required by an algorithm to complete its execution.
- The time and space complexity of an algorithm is also represented by notations such as the Big O notation

Algorithm complexity

Best case time complexity: A measure of the minimum time that the algorithm will require for an input of size 'n.'

Worst case time complexity: A measure of the maximum time that the algorithm will require for an input of size 'n.'

Average case time complexity: The time that the algorithm will require to execute a typical input data of size 'n.'

ALGORITHM COMPLEXITY: BIG-OH NOTATION

When resolving a computer-related problem, there will generally be various solutions.

These individual solutions will often be in the shape of different algorithms having different logic, so:

- we normally want to compare the algorithms to see which one is more efficient.
- The Big O analysis provides some basis for computing and measuring the efficiency of a specific algorithm.

Algorithm complexity: Big-Oh notation and growth rate

When two algorithms are compared to know which is more efficient, the most important factor to consider is the growth rate of both algorithms.

The Big O notation is very useful because it describes the growth rate of algorithms.

Example:

Let's consider two functions (1000N and N^2), to see which one grows faster (is less efficient)

ALGORITHM COMPLEXITY: BIG-OH NOTATION AND GROWTH RATE

The common computational complexity of algorithms by growth rate are given below

Function	Name
C	Constant
Log N	Logarithmic
Log2 N	Log-squared
N	Linear
N Log N	
N2	Quadratic
N3	Cubic
2N	Exponential

Algorithm complexity: time complexity

O(1) - Time complexity of a function (or set of statements) is considered as O(1) if it doesn't contain loop, recursion and call to any other non-constant time function.

O(n) - Time Complexity of a loop is considered as O(n) if the loop variables is incremented/decremented by a constant amount.

$O(n^c)$ - Time complexity of nested loops is equal to the number of times the innermost statement is executed.

ALGORITHM COMPLEXITY: TIME COMPLEXITY

O(Log n) - Time Complexity of a loop is considered as O(Log n) if the loop variables is divided/multiplied by a constant amount.

O(Log Log n) - Time Complexity of a loop is considered as O(Log Log n) if the loop variables is reduced/increased exponentially by a constant amount.

$O(2^n)$ - Algorithms with running time $O(2^n)$ are often recursive algorithms that solve a problem of size n by recursively solving two smaller problems of size n-1.

Algorithms characteristics

- Algorithms should have input values that will be processed to produce some output.
- Algorithms should have straightforward and clear instructions.
- Algorithms should have limited and countable number of instructions.
- Algorithms should not be dependent on any programming language.

Algorithms: Why We Need Them

If we are able to break down the solution to small and sizable problems into smaller steps as required by algorithms, we can easily optimise the performance of the solution.

We need algorithms for reasons such as performance and scalability.

Algorithms design considerations

- Algorithms should be modular such that we should be able to break down the solution to the problem into smaller steps.
- Algorithms should be correct.
- Algorithms should be maintainable and easily refactored.
- Algorithm steps should be logical and practical.

ALGORITHMS DESIGN CONSIDERATIONS

- Algorithms should be extensible such that another algorithm programmer or designer can update and upgrade it.
- Algorithms should be robust such that the solution to the problem is clearly defined.
- Algorithms should be simple such that it is very easy to understand.

FURTHER READING RESOURCES

- Cmelik, B., Sahni, S. (1995). Software Development in C. United States: Silicon Press.
- Chaudhuri, A. B. (2020). Flowchart and algorithm basics: The art of programming. Mercury Learning and Information.
- Chaudhuri, A. B. (2005). The Art of Programming Through Flowcharts & Algorithms. Firewall Media.

REFERENCES

1. *Chaudhuri, A. B. (2020). Flowchart and algorithm basics: The art of programming. Mercury Learning and Information.*
2. *Chaudhuri, A. B. (2005). The Art of Programming Through Flowcharts & Algorithms. Firewall Media.*

THANK YOU